

Двумерные массивы в C++

- Допустим, необходимо обработать некоторые данные из таблицы.
- В таблице есть две характеристики:
количество строк и количество столбцов.
- Также и в двумерном массиве, кроме количества элементов массива, есть такие характеристики как, количество строк и количество столбцов двумерного массива.
- Визуально, двумерный массив — это таблица, со строками и столбцами.
- Фактически двумерный массив — это одномерный массив одномерных массивов.

Индексация элементов в двумерном массиве

	столбец 0	столбец 1	столбец 2	столбец 3
строка 0	arr[0][0]	arr[0][1]	arr[0][2]	arr[0][3]
строка 1	arr[1][0]	arr[1][1]	arr[1][2]	arr[1][3]
строка 2	arr[2][0]	arr[2][1]	arr[2][2]	arr[2][3]

- Классический труд «Язык программирования C» Б. Кернигана и Д. Ритчи гласит «В языке C есть возможность работать с многомерными прямоугольными массивами, хотя на практике они используются гораздо реже, чем массивы указателей».
- C++ практически полностью унаследовал работу с многомерными массивами своего предка.



Описание

- <тип элементов> <имя массива> [количество строк][количество столбцов];
- Пример:

```
int a[5][3];
```

Инициализация

- При статической (определяемой на этапе компиляции) инициализации значения C-массива перечисляются в порядке указания размеров (индексов) в определении массива.
- Каждый уровень (индекс), кроме самого младшего, многомерного массива заключается в свою пару фигурных скобок. Значения самого младшего индекса указываются через запятую.

Пример инициализации

```
const unsigned int DIM1 = 3;  
const unsigned int DIM2 = 5;  
  
int ary[DIM1][DIM2] = {  
    { 1, 2, 3, 4, 5 },  
    { 2, 4, 6, 8, 10 },  
    { 3, 6, 9, 12, 15 }  
};
```

Заполнение массива значениями

- Многомерный массив заполняется значениями с помощью **вложенных циклов**.
- Как правило, количество циклов совпадает с размерностью массива.

```
for (int i = 0; i < DIM1; i++)  
{  
    for (int j = 0; j < DIM2; j++) {  
        ary[i][j] = (i + 1) * 10 + (j + 1);  
    }  
}
```


Расположение в памяти

- Для двумерного C-массива выделяется единый блок памяти необходимого размера:
- *число байт = размер второго измерения * *размер первого измерения * *sizeof (базовый тип)*
- Имя (идентификатор) двумерного C-массива **является указателем на первый элемент массива** (так же как и для одномерных массивов).

Передача в функцию

- Когда двумерный массив используется как аргумент функции, передается указатель на первый элемент.
- Функция, получающая двумерный массив, должна, как минимум, определять размер первого измерения, поскольку компилятору необходимо знать длину каждой строки для корректной индексации массива.

Создание и уничтожение динамических многомерных массивов

```
int **a;      // (1)
```

```
    // создание
```

```
a = new int* [m]; // массив указателей (2)
```

```
for (int i = 0; i < m; i++) {    // (3)
```

```
a[i] = new int[n]; // инициализация указателей  
}
```

Порядок действий при создании динамического двумерного массива

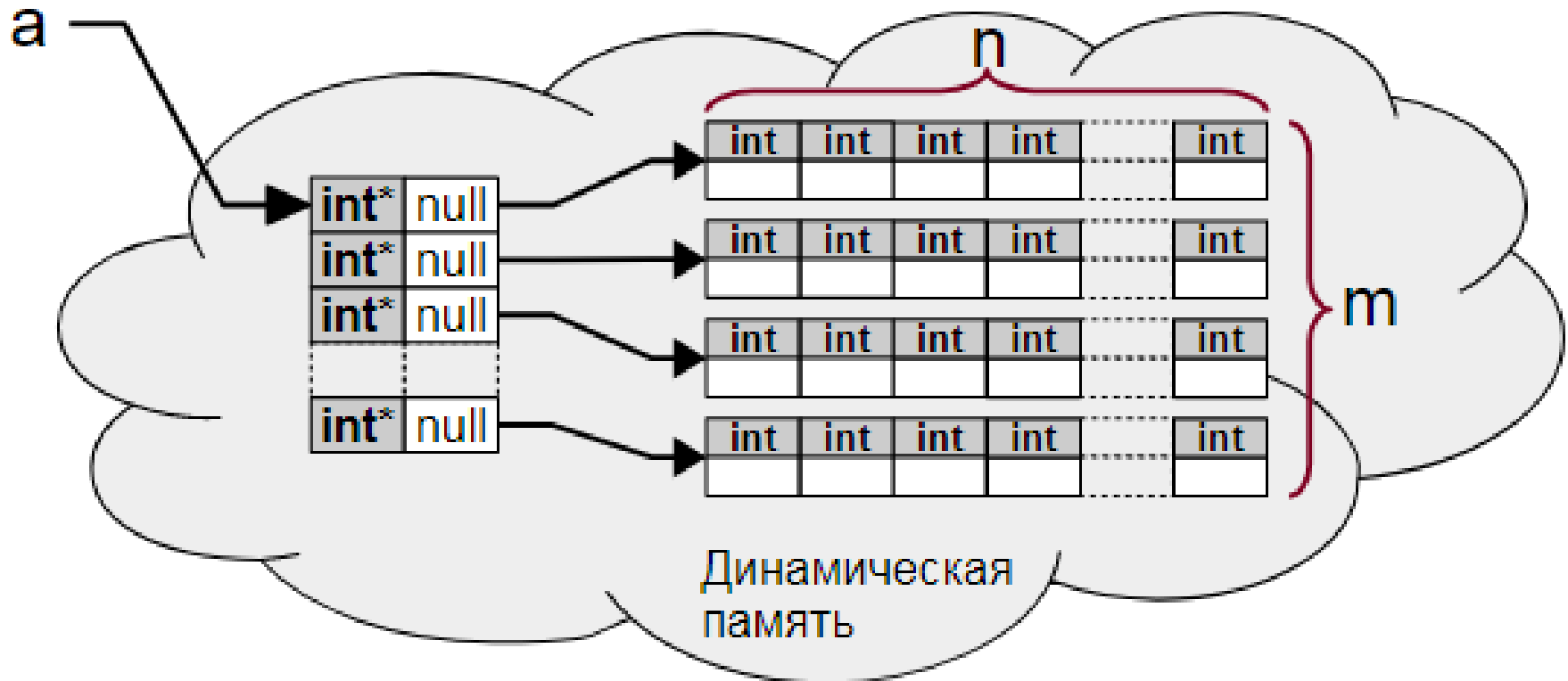
(1) Для доступа к двумерному массиву объявляется переменная типа указатель на указатель на тип (в примере это указатель на указатель на `int`).

(2) Переменная инициализируется оператором **`new`**, который выделяет память для массива указателей на `int`.

(3) В цикле каждый элемент массива указателей инициализируется оператором **`new`**, который выделяет память для массива типа `int`.

Схема выделения памяти

```
for(int i = 0; i < m; i++)  
    a[i] = new int[n];
```



Удаление

- Освобождение памяти происходит строго в обратном порядке: сначала уничтожаются массивы значений типа `int`, а затем уничтожается массив указателей.

```
// уничтожение
    for (int i = 0; i < m; i++) {
        delete[] a[i];
    }
delete[] a;
```

- Работа с динамическим многомерным массивом синтаксически полностью совпадает с работой с многомерным C-массивом.
- Но между этими массивами есть глубокое различие.

1) для динамического массива выделяется другой объём памяти.

- Если посчитать, сколько памяти будет выделяться для двумерного массива из примера выше, то получится:
первый оператор new выделил память для 3 указателей,
второй оператор new в цикле трижды выделил память для 5 элементов типа int. ***Т.е. получилось, что выделили памяти для 15 значений типа int и для 3 значений типа указатель на int.***
- Для C-массива компилятором была выделена память ***только для 15 значений типа int.***

2) память, выделенная для динамического массива, не непрерывна.

Следовательно, обращение с двумерным массивом как с одномерным работать не будет.

3) передача многомерных массивов в функции и работа с ними будет отличаться для динамических массивов и С-массивов.

- **Динамический многомерный массив реализуется как массив указателей на массивы, значения в которых, в свою очередь, тоже могут быть указателями на массивы.**
- **Последним звеном в этой цепочке всегда будут массивы со значениями целевого типа.**
- **Динамический многомерный массив НЕ является С-массивом.**

- Массив указателей на массивы — структура более гибкая, чем двумерный C-массив.
- Например, для массива указателей на массивы размеры массивов могут быть разными, или какой-то массив может вообще отсутствовать.
- Наиболее распространённым примером является «массив строк», т.е. массив указателей на массивы типа **char**.

Пример 1. Определение и инициализация двумерного C-массива

```
char month[12][10] = {  
    "January", "February",  
    "March",  
    "April", "May", "June",  
    "July", "August", "September",  
    "October", "November",  
    "December"  
};
```

Характеристики для примера 1

- Каждая C-строка занимает ровно 10 байт, включая завершающий ноль (считаем, тип `char` имеет размер 1 байт).
- Неиспользуемые байты у коротких строк, вроде «*Maу*», содержат «мусор».
- Весь массив занимает один непрерывный блок памяти размером 120 байт (12 строк по 10 символов).

Пример 2. Одномерный массив указателей на массивы

```
char *month[12] = {  
    "January", "February", "March",  
    "April", "May", "June",  
    "July", "August", "September",  
    "October", "November",  
    "December"  
};
```

Характеристики для примера 2

- Вся информация, доступная через переменную `month`, занимает 13 блоков памяти: массив из 12 указателей и 12 блоков памяти, адреса которых хранятся в указателях, содержащих C-строки с названиями месяцев.
- Нет никакой гарантии, что 12 блоков памяти с C-строками будут расположены в памяти последовательно и в порядке, соответствующем перечислению в инициализаторе.

Общее для двух реализаций

- В обоих случаях доступ к символу **b** в строке «**February**» будет осуществляться выражением `month[1][2]`.
- И т.д.